POO e os 4 Pilares

Material de apoio — Programação Orientada a Objetos em Java

> **Código-fonte da aplicação (entregável):** github.com/DiegaoDev2504/loja
> Aplicação Java de gerenciamento de clientes, produtos e pedidos que aplica os conceitos deste material.

O que é POO

Programação Orientada a Objetos é um jeito de organizar o código em **objetos**, cada um com seus dados e com as ações que pode fazer.

- **Classe:** o molde, que define atributos e métodos. Exemplo: `Produto`.
- **Objeto:** uma instância concreta da classe. Exemplo: `new ProdutoDigital(3, "Curso de Java", 199.00)`.
- **Atributos:** os dados do objeto (`nome`, `preco`).
- **Métodos:** o que o objeto faz (`calcularPrecoFinal()`).

1 Encapsulamento

Proteger os dados do objeto, para que só sejam acessados ou alterados por métodos controlados.

- Os atributos ficam `private`; o acesso é feito por `getters` e `setters`.
- Os setters podem **validar** os dados antes de aceitá-los.
- **Vantagem:** o objeto nunca fica num estado inválido.

```
private double preco;

public void setPreco(double preco) {
    if (preco < 0) throw new IllegalArgumentException("Preço inválido");
    this.preco = preco;
}
```

2 Herança

Uma classe **reaproveita** atributos e métodos de outra e pode acrescentar ou modificar comportamentos.

- Usa a palavra-chave `extends`.
- Representa uma relação **"é um"**: um *Cliente* **é uma** *Pessoa*.
- **Vantagem:** evita repetir código.

```
public class Cliente extends Pessoa { ... }
```

3 Polimorfismo

"Muitas formas": o mesmo método se comporta de forma diferente conforme o tipo real do objeto.

- **Sobrescrita** (`@Override`): a subclasse redefine um método da superclasse.
- **Sobrecarga**: métodos com o mesmo nome e parâmetros diferentes.
- **Vantagem:** o código trabalha com o tipo geral (`Produto`) sem precisar saber o tipo específico.

```
Produto p = new ProdutoDigital(...);
p.calcularPrecoFinal(); // executa a versão de ProdutoDigital
```

4 Abstração

Mostrar só o essencial e esconder os detalhes de implementação.

- Usa **classes abstratas** (`abstract class`) e **interfaces**.
- Define *o que* fazer e deixa *como* fazer para as subclasses.
- **Vantagem:** simplifica o uso e padroniza o comportamento.

```
public abstract class Produto {
    public abstract double calcularPrecoFinal();
}
```

Bônus: Composição

Um objeto é **formado por outros objetos**. É a relação **"tem um"**: um *Pedido* **tem** *Itens*.

- **Vantagem:** costuma ser mais flexível que a herança.

Resumo

Pilar	Ideia central	Palavra-chave em Java
Encapsulamento	Proteger os dados	`private`, getters/setters
Herança	Reaproveitar código	`extends`
Polimorfismo	Mesmo método, comportamentos diferentes	`@Override`
Abstração	Expor só o essencial	`abstract`, `interface`